untitled

File Edit Format Run Options Window Help

```
def dfs_recursive(graph, node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(node)
    order.append(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs_recursive(graph, neighbour, visited, order)
    return order

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    order = []
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                stack.append(neighbour)
    return order

def dfs_path(graph, start, goal, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    if start == goal:
        return [start]
    for neighbour in graph[start]:
        if neighbour not in visited:
            sub = dfs_path(graph, neighbour, goal, visited)
            if sub:
                return [start] + sub
    return None

if __name__ == "__main__":
    graph = {
        "A": ["B", "C"],
        "B": ["D", "E"],
        "C": ["F", "G"],
        "D": [], "E": [], "F": [], "G": [],
    }
    print("DFS (recursive) from A:", " -> ".join(dfs_recursive(graph, "A")))
    print("DFS (iterative) from A:", " -> ".join(dfs_iterative(graph, "A")))
```

Ln: 50 Col: 67

26°C Light rain

ENG IN 09:33 25-10-2026

```
DFS (recursive) from A: A -> B -> D -> E -> C -> F -> G
DFS (iterative) from A: A -> B -> D -> E -> C -> F -> G
A path from A to G    : A -> C -> G
DFS order from 1 (cyclic graph): [1, 2, 4, 3, 5]
```